

Single-pass cross-referencing in Lua¹TeX: hold and patch

Srikanth Mohankumar, Nareshkumar
Krishnaswamy

Abstract

The `monoref` package resolves a bounded set of $\text{\LaTeX}$ cross-references in one Lua¹TeX run: `\ref`, `\pageref`, a body-page total, and a generated table of contents. It does this by holding finished pages as Lua node lists, recording the true page of labels with invisible markers, and replacing reserved reference slots just before final shipout. This article describes the implementation and its trade-offs.

1 The two-pass problem

$\text{\LaTeX}$ cross-references are normally a file-based fixed point. On one run, commands such as `\label` write label values and page numbers to the `.aux` file.¹ On the next run, `\ref` and `\pageref` read those values back. If the new values differ from the old ones, another run may be needed. Build tools such as `latexmk` automate that loop, but the document still depends on state from a previous compilation.

The difficulty is especially clear for a forward reference:

```
\section{Intro}\label{sec:intro}
See Section~\ref{sec:res}.
...
\section{Results}\label{sec:res}
```

When the first paragraph is broken into lines, the value of `sec:res` has not yet been seen. In standard $\text{\LaTeX}$, the earlier run supplies the answer. `monoref` instead delays the commit: it lets $\text{\TeX}$ build pages, but it does not write them to the PDF file until the end of the document.

The scope is deliberately limited. The package is not a general replacement for the `.aux` file. It covers numbered labels, page references, `\lastpage`, and a contents list for `\section`, `\subsection`, and `\subsubsection`. The constraint makes a one-run implementation practical.

2 Pipeline

The implementation is a hold-and-patch pipeline. Figure 2 shows the order. Pages produced by the page builder are copied into Lua tables.² As each

¹ The `.aux` file is also used for many other document-level facts; `monoref` only addresses the reference subset described here.

² The page is kept as a Lua node list, not rendered to an intermediate image or external file.

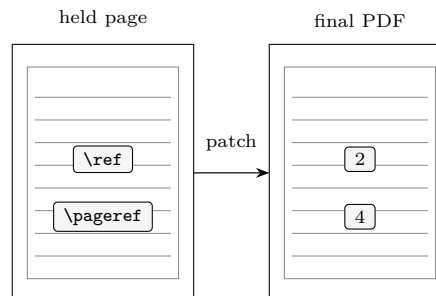


Figure 1: A reader sees ordinary references; internally the early page contains attributed slots until finalization.

page is held, a marker scan assigns labels and TOC entries to the page on which they actually appear. After the input stream ends, all slots are patched, lines containing slots are repacked, the generated contents pages are added ahead of the body, and the queued pages are shipped for real.

3 Holding pages

The package uses the $\text{\LaTeX}$ `shipout/before` hook. During normal processing the hook copies the current shipout box, scans the copy, stores it, and discards the original. During final flushing a guard prevents the same hook from intercepting the pages again.³

The TeX side is short:

```
\AddToHook{shipout/before}{%
  \ifmonoref@flushing\else
    \directlua{monoref.hold(
      \number\ShipoutBox,
      \the\value{page})}%
    \DiscardShipoutBox
  \fi}
```

The corresponding Lua function copies the box node list and stores it in either the body-page table or the contents-page table:

```
function monoref.hold(reg, pg)
  local b = tex.getbox(reg)
  if not b then return end
  local c = node.copy(b)
  scan(c.head, pg)
  local dest = (monoref.target == "toc")
    and monoref.toc_pages
    or monoref.held
  dest[#dest + 1] = c
end
```

The two tables matter because the contents pages are generated only at the end and then prepended

³ The guard is essential: without it, the final shipouts would be held again instead of being written to the output file.

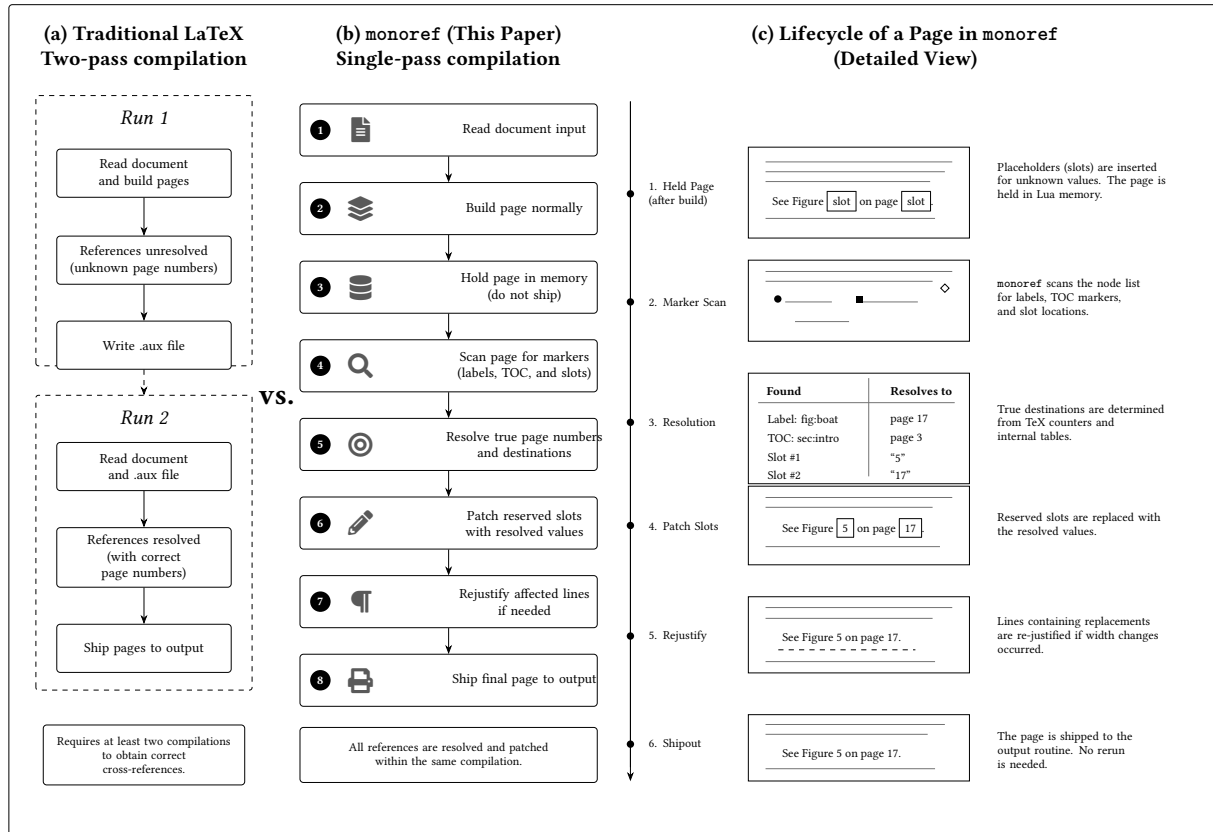


Figure 2: Schematic comparison of the ordinary two-pass reference loop and the single-pass `monoref` pipeline. The image is an overview: in the implementation, label and TOC markers are stamped when a page is held, while slot boxes are found later during final patching.

to the body. Body page totals are computed from `#monoref.held`, so generated front matter does not change `\lastpage`.

4 Reserved slots

A not-yet-known value cannot be inserted as text. Instead, `monoref` emits an attributed `\hbox` of a predictable width. The default templates are:

```
\monorefreftemplate % for \ref
\monorefpagetabletemplate % for pages
```

A positive `\monorefslotwidth` overrides both templates.

This is the actual slot constructor, with line breaks added for the column:

```
\newcommand\monoref@slot[2]{%
  \begingroup
  \ifdim\monorefslotwidth>\z@
    \monoref@tmpwd=\monorefslotwidth
  \else
    \settowidth\monoref@tmpwd{#2}%
  \fi
  \monoref@slotattr=#1\relax
  \hbox to \monoref@tmpwd{\hfil}%
\endgroup}
```

```
\fi
\monoref@slotattr=#1\relax
\hbox to \monoref@tmpwd{\hfil}%
\endgroup}
```

The package assigns the `monorefslot` Lua attribute to the placeholder. At flush time the Lua backend finds attributed boxes and replaces their contents.

There is one important distinction. A backward `\ref` can be printed immediately, because the target label has already registered its section or equation number. A forward `\ref` becomes a slot. A `\pageref` is always a slot: a label's true page is known only after the page builder has assembled a page.

5 Finding true pages

Reading `\value{page}` at `\label` time is not reliable, because TeX's page builder is asynchronous. Material being read now may eventually be moved to the next page. `monoref` therefore records page numbers at shipout time, not at label time.

The TeX marker is an empty special carrying a Lua attribute:

```
\newcommand\monoref@mark[1]{%
  {\monoref@markattr=#1\relax \special{}}}
```

`\label` stores the current label value, makes a Lua marker record, drops the marker into the list, and calls the original `\label`. TOC records use the same mechanism. The page copy is then scanned recursively:

```
local m = node.get_attribute(n,
  monoref.markattr)
if m and monoref.marks[m] then
  local r = monoref.marks[m]
  if r.kind == "label" then
    monoref.labels[r.key].page = pg
  elseif r.kind == "toc" then
    monoref.toc[r.key].page = pg
  end
end
end
```

The marker has no visible output. Its only purpose is to survive into the page node list so the final page assignment is made from the same object that will later be shipped.

6 Patching and re-justifying

At finalization, each slot is replaced by glyph nodes for the resolved value. The backend records the current font when a slot is created, then uses that font when constructing the replacement glyphs.

The placeholder does not remain at its reserved width. The Lua code packs the replacement at its natural width:

```
local packed = node.hpack(gf or node.new(GLUE))
hbox.head = packed.head
hbox.width = packed.width
hbox.height = packed.height
hbox.depth = packed.depth
```

After a direct child of a line has been filled, the line is packed again to its original width:

```
local packed =
  node.hpack(hbox.head, hbox.width,
    "exactly")
hbox.head = packed.head
hbox.glue_set = packed.glue_set
hbox.glue_sign = packed.glue_sign
```

This is the reason the reserved template is only a line-breaking estimate. It keeps the original paragraph break from needing to change; after patching, interword glue absorbs the space freed by shorter

values. If the final value is wider than the reserved template, the line can still overrun. The package exposes the templates because only the document author knows the largest expected forward value.

7 Contents and links

The generated table of contents is collected by redefining `\section`, `\subsection`, and `\subsubsection`. For each numbered heading, the package stores the level, number, title, and a marker whose page is filled in by the same shipout scan used for labels. At `\end{document}`, it switches `monoref.target` to `toc`, selects roman page numbering, typesets the contents list, and holds those pages separately.

TOC lines are intentionally simple:

```
\newcommand\monoref@tocline[5]{%
  \par\noindent\monoref@indent{#2}%
  \monoref@link{toc:#1}{%
    \makebox[2.6em][l]{#3#4}%
  } \dotfill #5}
```

The body retains arabic page numbering, and the generated contents pages are placed before it in the final queue. This avoids a circular dependency: adding the contents page does not renumber the body.

When `hyperref` is loaded before `monoref`, references and contents entries become links.⁴ The package creates its own named destinations:

```
\newcommand\monoref@target[1]{%
  \ifmonoref@hyper
    \hypertarget{monoref:#1}{}%
  \fi}
\newcommand\monoref@link[2]{%
  \ifmonoref@hyper
    \hyperlink{monoref:#1}{#2}%
  \else #2\fi}
```

The links do not depend on `hyperref`'s `.aux`-based anchors, so they can be created in the same single run as the reference values.

8 Finalization

The last step starts by setting the body-page total and patching both page tables:

```
function monoref.finalize()
  monoref.total = #monoref.held
  for i = 1, #monoref.toc_pages do
    patch_box(monoref.toc_pages[i])
  end
  for i = 1, #monoref.held do
    patch_box(monoref.held[i])
  end
end
```

⁴ The load order matters because `monoref` replaces `\ref`, `\pageref`, and the sectioning commands after detecting whether `hyperref` is already present.

```
end
end
```

The same function then appends `toc_pages` and `held` to `monoref.queue`, in that order. The TeX end-document hook then pops each page into a box register and ships it while the flushing guard is true. The final PDF is therefore produced by ordinary `\shipout`; the unusual part is that the pages being shipped were assembled earlier.

9 A minimal example

The package is loaded after `hyperref` if links are wanted:

```
\documentclass{article}
\usepackage[hidelinks]{hyperref}
\usepackage{monoref}
\begin{document}
\section{First}\label{sec:first}
See Section-\ref{sec:second} on
page-\pageref{sec:second}.
This document has \lastpage\ body page(s).
\section{Second}\label{sec:second}
Back to Section-\ref{sec:first}.
\end{document}
```

The example is compiled once with Lua $\text{\LaTeX}$. The distributed `examples/` directory includes variants for a generated TOC, the `notoc` option, `amsmath` equation labels, `fancyhdr` page totals, and custom slot widths.

10 Trade-offs

The design is intentionally small, but the costs are direct. `monoref` holds the whole document in memory until `\end{document}`. It also intercepts every shipout, so it is not a good companion for packages that perform independent shipout manipulation.

The user-level interface is implemented by redefining the label and reference commands, together with the three sectioning commands used for the TOC. Numbered equation references work, including with `amsmath`, because the package also updates `\ltx@label` when `amsmath` is present. More elaborate reference interfaces — `\eqref`, `\autoref`, `\nameref`, `\cleveref`, and `\varioref` — are outside the current implementation. Lists of figures and tables are not generated.

Finally, the slot mechanism preserves the original line break; it does not recompute paragraph breaking. A forward value wider than the reserved template can overflow. This is the central trade-off in the package: one run is obtained by postponing

page output, not by rerunning the paragraph builder after all values are known.

11 Build-time implications

The practical benefit is avoiding a second full engine run for the supported reference set. A useful way to state the cost is to let T be the elapsed time of one ordinary Lua $\text{\LaTeX}$ run for a given document. A manual two-run build pays about $2T$. `latexmk` pays about kT plus a small driver overhead, where k is the number of runs needed for the auxiliary files to settle. With `monoref`, the expected cost is closer to one run plus the cost of copying pages, scanning markers, patching slots, and repacking affected lines.⁵

Workflow	Main cost
two manual runs	about $2T$
<code>latexmk</code>	about kT plus driver checks
<code>monoref</code>	about T plus hold/patch work

The saving can be more visible in production documents with many pages, large OpenType fonts, and heavy package stacks, because each additional run repeats macro expansion, font loading, shaping, page building, and shipout work. The package does not make one page cheaper to typeset; it removes the need to typeset the same supported-reference document a second time.

A timing table for a particular production document should compare the same source tree from a clean output directory. The useful rows are: one Lua $\text{\LaTeX}$ run with `monoref`; two explicit Lua $\text{\LaTeX}$ runs for the conventional workflow; and a `latexmk` build using the same engine and options. The last row is important because it records the time users actually pay in automated builds, including rerun detection and any extra passes caused by other auxiliary-file features. The table should report elapsed time, not just CPU time, because font loading and filesystem traffic are part of the publishing cost.

12 Availability and reproducibility

`monoref` is distributed through CTAN; the package page is listed with the other external resources below. The source distribution consists of a documented `.dtx` file, an installer, the Lua backend, a README, and runnable examples. The implementation described here corresponds to version 1.3, dated 2026-07-07.

The examples are intentionally small regression checks as well as user examples. They cover the minimal case, a generated TOC, `hyperref` links, the

⁵ These are cost models, not benchmark numbers. A publishable timing table should state the hardware, operating system, TeX Live version, engine command, page count, font set, and package set.

`notoc` option, `fancyhdr` page totals, and custom slot widths. In the source tree they can be compiled with one Lua $\text{\LaTeX}$ run each; no previous `.aux` file is needed for the features provided by the package.

The article source itself uses colorized listings only as a reading aid. The code remains ordinary text in a light frame, so the examples remain usable if printed without color. This follows the TUGboat preference that code should not rely on color alone.

13 Conclusion

`monoref` shows that a useful part of $\text{\LaTeX}$'s cross-reference problem can be solved inside one Lua $\text{\LaTeX}$ run. The method is mechanical: hold pages, mark true destinations, reserve slots, patch node lists, re-justify affected lines, and ship the stored pages. It is not a universal cross-reference system, but for documents within its stated scope it replaces the usual rerun loop with a single end-of-document patching phase.

References

- [1] Leslie Lamport. *L $\text{\LaTeX}$: A Document Preparation System*. Addison-Wesley, second edition, 1994.
- [2] Lua $\text{\LaTeX}$ development team. *Lua $\text{\LaTeX}$ Reference Manual*. <https://ctan.org/pkg/luatex>
- [3] Sebastian Rahtz and Heiko Oberdiek. *The `hyperref` package*. <https://ctan.org/pkg/hyperref>
- [4] John Collins. *l $\text{\LaTeX}$ mk: Fully automated L $\text{\LaTeX}$ document generation*. <https://ctan.org/pkg/latexmk>
- [5] $\text{\LaTeX}$ Project Team. *The L $\text{\LaTeX}$ shipout hooks*. Part of the $\text{\LaTeX}$ kernel documentation.
- [6] CTAN. *monoref package page*. <https://ctan.org/pkg/monoref>

◇ Srikanth Mohankumar
TNQ Tech Pvt. Ltd.
Chennai, Tamilnadu, India
srikanth.mohan@tnqtech.com

◇ Nareshkumar Krishnaswamy
TNQ Tech Pvt. Ltd.
Chennai, Tamilnadu, India
nareshkumar.k@tnqtech.com